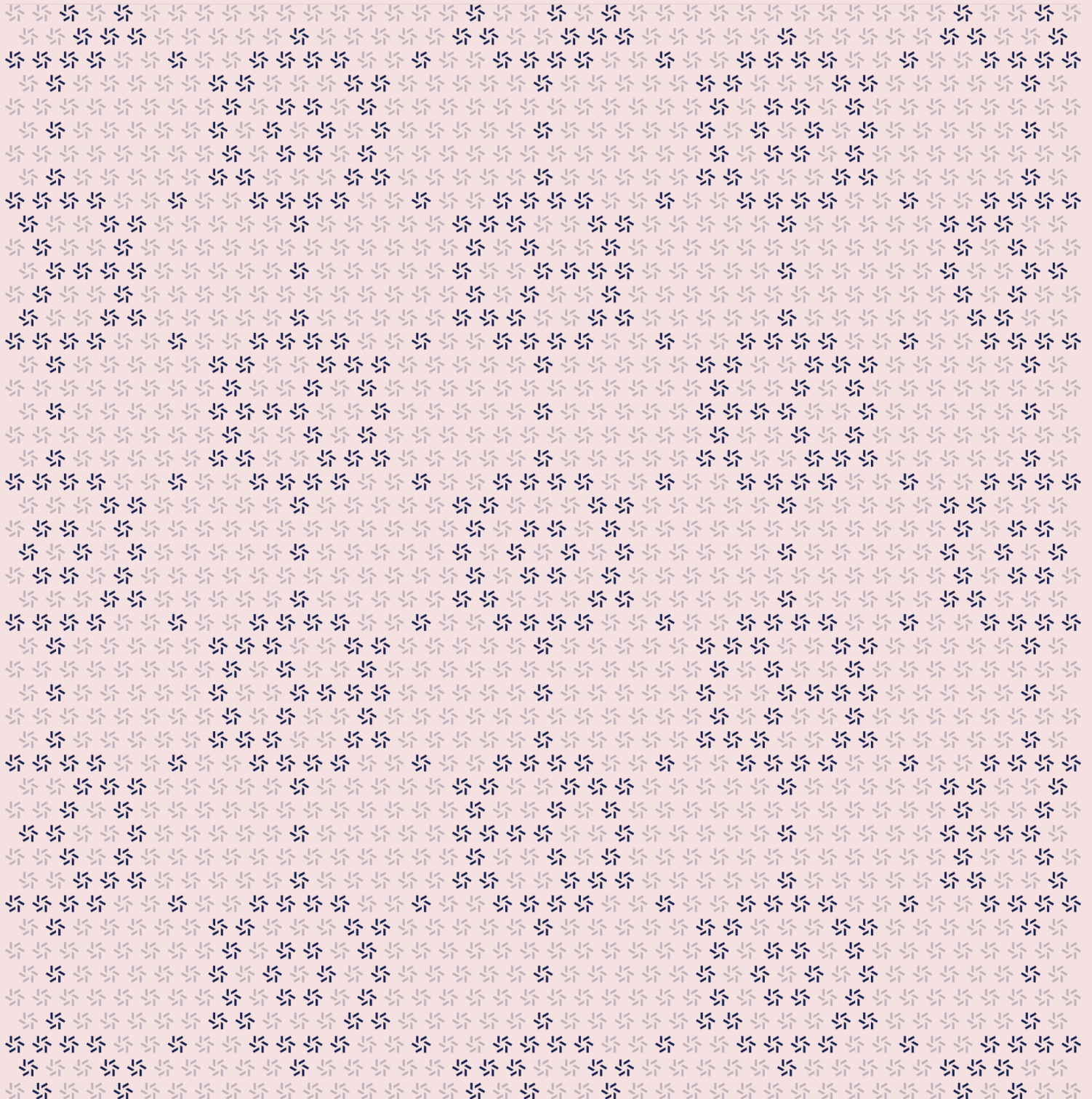


March 11, 2026

LayerZero Stellar

Stellar Application Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About LayerZero Stellar	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	12
3.1. Footprint ledger entry limit results in denial of service	13
3.2. Outdated Stellar Soroban SDK version could result in issues with function wiring	15
3.3. Inconsistent behaviors across chains	17
4. Discussion	19
4.1. Lack of withdrawal in Counter OApp implementation	20
4.2. Duplicate signer logic	20
4.3. Non-root authentication	21

4.4.	Scope-change details	21
5.	System Design	23
5.1.	Key differences between LayerZero Stellar and EVM implementations	24
6.	Assessment Results	29
6.1.	Disclaimer	30

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for LayerZero Labs from January 28th to February 26th, 2026. During this engagement, Zellic reviewed LayerZero Stellar's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any vulnerabilities that could result in the loss of user funds?
 - Are access controls implemented effectively to prevent unauthorized operations?
 - Are there any discrepancies with the EVM implementation?
 - Are there any Stellar-Soroban-specific constraints that need consideration?
 - Are there any ways developers could render themselves vulnerable unintentionally due to macro usage?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Off-chain components
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped LayerZero Stellar files, we discovered three findings. No critical issues were found. One finding was of high impact, one was of medium impact, and the remaining finding was informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of

LayerZero Labs in the Discussion section ([4](#), [7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	1
Medium	1
Low	0
Informational	1



2. Introduction

2.1. About LayerZero Stellar

LayerZero Labs contributed the following description of LayerZero Stellar:

LayerZero works with asset issuers, chains, and applications to make money more extensible. Issue any asset on chain, extend its reach to new ecosystems, connect fragmented liquidity and more.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the files.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped files itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

LayerZero Stellar Files

Type	Rust
Platform	Stellar-Soroban
Target	monorepo-internal
Repository	https://github.com/LayerZero-Labs/monorepo-internal ↗
Version	8f68242a42bfa5556f04e68bc70cbab4c658fc88
Programs	endpoint-v2 common-macros utils message-libs/blocked-message-lib message-libs/message-lib-common message-libs/uIn-302 message-libs/treasury workers/worker workers/dvn workers/fee-lib-interfaces oapps/oapp oapps/oapp-macros oapps/oft oapps/oft-core

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 5.7 person-weeks. The assessment was conducted by two consultants over the course of 4.4 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✧ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Ethan Lee
✧ Engineer
ethl@zellic.io ↗

Ayaz Mammadov
✧ Engineer
ayaz@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

January 28, 2026	Start of primary review period
February 12, 2026	Scope change from 8f68242a to b659ed83
February 16, 2026	Scope change from b659ed83 to 00fca66e
February 26, 2026	End of primary review period
March 11, 2026	Start of secondary review period
March 11, 2026	Scope change from 00fca66e to 2ed91ed3
March 12, 2026	Scope change from 2ed91ed3 to fe230189
March 12, 2026	Scope change from fe230189 to cf2ddcfa
March 14, 2026	End of secondary review period
March 23, 2026	Start of tertiary review period
March 23, 2026	Scope change from cf2ddcfa to fe2f3954
March 24, 2026	Scope change from fe2f3954 to 3822cc60
March 25, 2026	End of tertiary review period
March 26, 2026	Start of quaternary review period
March 27, 2026	Scope change from 3822cc60 to 3c78439e
March 27, 2026	End of quaternary review period
April 13, 2026	Start of quinary review period
April 13, 2026	Scope change from 3c78439e to e365dad3
April 13, 2026	End of quinary review period

April 29, 2026 Start of senary review period

April 29, 2026 The repository was migrated to [monorepo-external ↗](#)

April 29, 2026 Scope change from [e365dad3 ↗](#) to [c1fb16ec ↗](#) in monorepo-external

April 29, 2026 End of senary review period

July 14, 2026 Start of septenary review period

July 14, 2026 Scope change from [c1fb16ec ↗](#) to [d20f6fa2 ↗](#)

July 14, 2026 End of septenary review period

3. Detailed Findings

3.1. Footprint ledger entry limit results in denial of service

Target	EndpointV2		
Category	Business Logic	Severity	High
Likelihood	High	Impact	High

Description

The Stellar network has a limit of 200 footprint ledger entries per transaction; in practical terms, any transaction that touches over 200 different storage slots will not be able to execute successfully.

One potential area this could happen is in the function `inbound_nonce`:

```
fn inbound_nonce(env: &Env, receiver: &Address, src_eid: u32, sender:
    &BytesN<32>) -> u64 {
    let mut nonce_cursor = Self::lazy_inbound_nonce(env, receiver, src_eid,
        sender);

    // Find the effective inbound current nonce
    while EndpointStorage::has_inbound_payload_hash(env, receiver, src_eid,
        sender, nonce_cursor + 1) {
        nonce_cursor += 1;
    }
    nonce_cursor
}
```

When attempting to skip nonces, the next unverified message must be found. However, in the process of checking the inbound nonces, there is a chance that, if there are more than 200 verified nonces, this function will error as it would require more ledger footprint entries than allowed in a Stellar transaction.

Impact

Depending on the OApp and implementations, this could result in a denial of service of an OApp.

Recommendations

Adjust the nonce system to account for this limit.

We also would recommend, anytime there is a loop or functional iteration going on, to attempt to

check this limit with tests if possible. That is, if a code path may be suspect of having a dynamic number of footprint entries, it is advisable to check this limit with tests.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in [PR #2826](#). They intend to merge these changes into the production branch.

3.2. Outdated Stellar Soroban SDK version could result in issues with function wiring

Target	Project-wide		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

At the time of writing, the Stellar Soroban version being used is V25.1.0. This version is affected by Stellar security advisory [GHSA-4chv-4c6w-w254](#).

The `#[contractimpl]` macro contains a bug in how it wires up function calls.

In Rust, functions on a type can be defined in two ways:

1. Directly on the type as an inherent function

```
impl MyContract {
    fn value() { ... }
}
```

2. Through a trait

```
impl Trait for MyContract {
    fn value() { ... }
}
```

These are two separate functions that happen to share the same name. Rust has rules for which one is called. When writing `MyContract::value()`, Rust always picks the one defined directly on the type, not the trait version.

Impact

As there is a lot of macro usage as well as exposed macros, there is a risk that developers writing OApps, or developers working on LayerZero, encounter the edge case associated with the published vulnerability.

Recommendations

Upgrade to a more secure and up-to-date version of Stellar Soroban SDK.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in [PR #2812 ↗](#). They intend to merge these changes into the production branch.

3.3. Inconsistent behaviors across chains

Target	Project-wide		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

At the time of writing, the implementation exposes behavioral differences in validation and quote semantics across EVM and Stellar Soroban implementations.

The first inconsistency appears in delegate validation during OApp initialization. EVM OAppCore explicitly rejects address(0) in the constructor. In Stellar, initialize_oapp accepts Option<Address> and forwards it directly to endpoint set_delegate, so initialization allows an unset delegate (None).

```
// contracts/protocol/stellar/contracts/oapps/oapp/src/oapp_core.rs#L16-L25
pub fn initialize_oapp<T: OAppCore + OwnableInitializer>(
    env: &Env,
    owner: &Address,
    endpoint: &Address,
    delegate: &Option<Address>,
) {
    T::init_owner(env, owner);
    OAppCoreStorage::set_endpoint(env, endpoint);
    LayerZeroEndpointV2Client::new(env,
    endpoint).set_delegate(&env.current_contract_address(), delegate);
}
```

```
// LayerZero-v2/packages/layerzero-
v2/evm/oapp/contracts/oapp/OAppCore.sol#L26-L31
constructor(address _endpoint, address _delegate) {
    endpoint = ILayerZeroEndpointV2(_endpoint);

    if (_delegate == address(0)) revert InvalidDelegate();
    endpoint.setDelegate(_delegate);
}
```

The second inconsistency is quote limit. OFT Core encodes amount as u64 / uint64, so the effective upper bound is u64::MAX. EVM quoteOFT reports this explicitly, but Stellar __quote_of_t uses i128::MAX, while to_sd still enforces u64::MAX. This creates a direct

precheck-versus-runtime-limit mismatch.

```
// contracts/protocol/stellar/contracts/oapps/oft-core/src/oft_core.rs#L169-L175
fn __quote_oft(env: &Env, send_param: &SendParam) -> (OFTLimit,
Vec<OFTFeeDetail>, OFTReceipt) {
    assert_with_error!(env, send_param.amount_ld >= 0,
OFTError::InvalidAmount);
    let limit = OFTLimit { min_amount_ld: 0, max_amount_ld: i128::MAX };
    let fee_details = vec![env];
    let oft_receipt = Self::__debit_view(env, send_param.amount_ld,
send_param.min_amount_ld, send_param.dst_eid);
    (limit, fee_details, oft_receipt)
}

// contracts/protocol/stellar/contracts/oapps/oft-core/src/utils.rs#L15-L18
pub fn to_sd(env: &Env, amount_ld: i128, conversion_rate: i128) -> u64 {
    let amount_sd = amount_ld / conversion_rate;
    assert_with_error!(env, amount_sd <= u64::MAX as i128,
OFTError::Overflow);
    amount_sd as u64
}
```

```
// LayerZero-v2/packages/layerzero-v2/evm/oapp/contracts/oft/OFTCore.sol#L114-L116
function quoteOFT(
    SendParam calldata _sendParam
)
returns (OFTLimit memory oftLimit, OFTFeeDetail[] memory oftFeeDetails,
OFTReceipt memory oftReceipt)
{
    // [...]
    uint256 minAmountLD = 0; // Unused in the default implementation.
    uint256 maxAmountLD = type(uint64).max; // Unused in the default
implementation.
    oftLimit = OFTLimit(minAmountLD, maxAmountLD);
    // [...]
}
```

Impact

Cross-chain operators and SDKs can misassume protocol invariants when implementations across different chains express constraints differently. In practice, this can lead to precheck and runtime mismatch, failed transaction attempts, and incorrect policy assumptions in automation layers.

Recommendations

We recommend the following.

1. Require an explicit Address instead of `Option<Address>` for delegate initialization.
2. Use `u64::MAX` instead of `i128::MAX` in quote limits.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in [PR #2663 ↗](#). They intend to merge these changes into the production branch.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Lack of withdrawal in Counter OApp implementation

The Counter OApp logic is out of scope; however, we noticed a difference in the implementation of Counter between the EVM reference and the Stellar version.

As a result, any refund from funds/fees will be stuck in the Counter OApp. This differs from the EVM implementation, which includes a `withdraw` function where authorized roles can withdraw accidental/received funds. We recommend adding the relevant `withdraw` to the Counter implementation.

4.2. Duplicate signer logic

The `add_signer` function disallows adding the same keys to the multi-sig, though it seems that two different public keys can be generated from the same private key, in which case this check would not be effective. However, it seems that the purpose of this validation is to prevent accidental inclusion of the same signer twice and not to guarantee one-signer-to-one-user correspondence.

```
fn add_signer(env: &Env, signer: &BytesN<20>) {
    // Not allowed to add zero address as signer
    assert_with_error!(env, signer != &BytesN::from_array(env, &[0u8; 20]),
        MultiSigError::InvalidSigner);
    // Not allowed to add same signer twice
    let mut signers = MultiSigStorage::signers(env);
    assert_with_error!(env, !signers.iter().any(|s| &s == signer),
        MultiSigError::SignerAlreadyExists);

    // Add signer to list
    signers.push_back(signer.clone());
    MultiSigStorage::set_signers(env, &signers);

    SignerSet { signer: signer.clone(), active: true }.publish(env);
}
```

4.3. Non-root authentication

On Stellar, when a public top-level method has no authentication and only performs cross-contract calls, the auth entries describe the subinvocations rather than the top-level call. An attacker in the mempool can front-run and hijack those auth entries to run only part of the intended logic — for example, in the upgrader, calling `upgrade` without `migrate` (leaving the contract in a bad state), or in the executor-helper, performing the value transfer without invoking `lz_receive` / `lz_compose` (leading to loss of user funds).

Note: This issue was discovered prior to its components' inclusion in scope on February 16th, 2026, and thus is out of scope for this engagement. We document it here for completeness. It has been fixed in [PR #2897](#).

4.4. Scope-change details

Here we outline the details of each scope change.

February 12th, 2026

A scope change occurred on February 12th, 2026. The change was from [8f68242a](#) to [b659ed83](#). Zellic conducted a diff audit between the previous scope and the new scope.

February 16th, 2026

A scope change occurred on February 16th, 2026. The change was from [b659ed83](#) to [00fca66e](#). Zellic conducted a diff audit between the previous scope and the new scope.

The following components were added to the scope:

- `contracts/protocol/stellar/contracts/upgrader/src`
- `contracts/protocol/stellar/contracts/workers/executor/src`
- `contracts/protocol/stellar/contracts/workers/executor-helper/src`

March 11th, 2026

A scope change occurred on March 11th, 2026. The change was from [00fca66e](#) to [2ed91ed3](#). Zellic conducted a diff audit between the previous scope and the new scope.

The following components were added to the scope:

- `contracts/protocol/stellar/contracts/oapps/sac-manager/src`

- `contracts/protocol/stellar/contracts/utis/src/rbac.rs`
- `contracts/protocol/stellar/contracts/common-macros/src/rbac.rs`

March 12th, 2026

Two scope changes occurred on March 12th, 2026.

The first change was from [2ed91ed3 ↗](#) to [fe230189 ↗](#). Zellic conducted a diff audit between the previous scope and the new scope.

The second change was from [fe230189 ↗](#) to [cf2ddcfa ↗](#). Zellic conducted a diff audit between the previous scope and the new scope.

March 23rd, 2026

A scope change occurred on March 23rd, 2026. The change was from [cf2ddcfa ↗](#) to [fe2f3954 ↗](#). Zellic conducted a diff audit between the previous scope and the new scope.

March 24th, 2026

A scope change occurred on March 24th, 2026. The change was from [fe2f3954 ↗](#) to [3822cc60 ↗](#). Zellic conducted a diff audit between the previous scope and the new scope.

March 27th, 2026

A scope change occurred on March 27th, 2026. The change was from [3822cc60 ↗](#) to [3c78439e ↗](#). Zellic conducted a diff audit between the previous scope and the new scope.

April 13th, 2026

A scope change occurred on April 13th, 2026. The change was from [3c78439e ↗](#) to [e365dad3 ↗](#). Zellic conducted a diff audit between the previous scope and the new scope.

April 29th, 2026

A scope change occurred on April 29th, 2026. The repository was migrated to [monorepo-external ↗](#). The change was from [e365dad3 ↗](#) in LayerZero-Labs/monorepo-internal to [c1fb16ec ↗](#) in LayerZero-Labs/monorepo-external. Zellic conducted a diff audit between the previous scope and the new scope.

July 14th, 2026

A scope change occurred on July 14th, 2026. The change was from [c1fb16ec ↗](#) to [d20f6fa2 ↗](#). Zellic conducted a diff audit between the previous scope and the new scope.

5. System Design

As part of the audit process, Zellic performed a technical comparison between the Stellar implementation of Endpoint V2, translated from the original Solidity/Move code, and its EVM counterpart. The goal was to identify vulnerabilities and bugs introduced during the translation as well as incorrect implementations that deviate from the protocol's specification.

This provides an overview of the differences between the Stellar and EVM implementations of LayerZero Endpoint V2.

5.1. Key differences between LayerZero Stellar and EVM implementations

This section describes how the Stellar (Soroban) implementation of LayerZero V2 differs structurally from the EVM reference implementation. Readers are assumed to be familiar with the EVM design.

1. Storage model

On EVM, storage is implicit and permanent. State variables occupy fixed slots, mappings hash their keys, and nothing expires.

On Stellar, storage is explicit and subject to ledger rent. Soroban exposes three tiers.

1. **Instance storage** — tied to contract lifetime, shared across all invocations. It is used for EID, token addresses, and library count.
2. **Persistent storage** — independent ledger entries with configurable TTLs, which must be actively extended. It is used for all messaging state, including nonces, payload hashes, library assignments, and delegate mappings.
3. **Temporary storage** — temporary entries that are gone forever when their TTL expires. It is used for a pending owner for two-step ownership transfer.

Storage access is mediated through a `#[storage]` macro that generates getter/setter methods. All access to a persistent entry automatically extends its TTL by a configured threshold if the remaining TTL falls below threshold. So active paths stay alive without manual intervention.

Unlike temporary storage, which is permanently deleted on expiry, *persistent* entries transition to an archived state when their TTL expires. The data is not lost and can be restored. On EVM, storage persists indefinitely at no ongoing cost.

2. Authorization for cross-contract calls

Soroban authorization is invocation scoped. When a caller authorizes a transaction, that authorization covers only the calls explicitly included in the caller's authorization entry. The endpoint cannot reuse or forward a caller's authorization to subcalls on its own. Each layer in the call chain must be independently authorized. This is different from the EVM's `msg.sender` propagation model, where the originating address flows implicitly through the call stack.

Here is an example of how authorization is required for token transfer.

```
pub fn clear_payload_and_transfer<T: OAppCore>(  
    // [...]  
    executor.require_auth(); // Authorization is also required here for token  
    transfer  
    if value != 0 {  
        let token_client = TokenClient::new(env,  
        &endpoint_client.native_token());  
        token_client.transfer(executor, &this_address, &value); // This  
        internally requires authorization from the executor  
    }
```

3. Authorization and authentication

On EVM, authorization uses `msg.sender` with a delegate mapping (`oapp -> delegate`). Workers use `AccessControl` roles.

On Stellar, authorization uses Soroban's `require_auth()` primitive and is abstracted by macros.

- `#[ownable]` (used by `LzExecutor`): This stores a single owner address in instance storage. Sensitive functions call `require_auth()` on that address.
- `#[multisig]` (used by `LzDVN`): The contract is self-owning. Actions require a threshold of `Secp256k1` signatures from Ethereum-format 20-byte addresses. Signature verification uses `Env::crypto().secp256k1_recover()`.
- `require_oapp_auth()` (used by the endpoint): This accepts either the `OApp` address or its registered delegate as the authorizer, matching the EVM delegate pattern.

The DVN's use of Ethereum-format addresses and `Secp256k1` signing is a deliberate design choice. DVN operators can reuse the same key material across EVM and Stellar deployments without managing separate key sets. The trade-off is that Stellar's native account model (`Ed25519`) is bypassed for DVN operations.

4. Upgradability strategy

The policy is identical between the two, but the mechanism differs.

On EVM, a proxy creates a permanent on-chain separation between the proxy address and the replaceable implementation address. The previous implementation remains deployed and its address is still readable on chain.

On Stellar, Stellar references new code by WASM hash (not deployment address), so the same WASM contract can be referenced by multiple contracts without redeployment. LayerZero implements the `Upgradeable` trait as follows.

- `upgrade(new_wasm_hash)` asserts the contract is not frozen and sets a `Migrating` flag in

instance storage, then directly replaces the contract's WASM at the ledger entry level.

- **migrate(migration_data)** is called after the upgrade, verifies the Migrating flag is set, runs the contract-specific `__migrate()` hook, and then clears the flag. Contracts using `upgradeable(no_migration)` get a no-op `__migrate` but still must call `migrate` to clear the flag.
- **freeze()** irreversibly sets a Frozen flag in instance storage, permanently blocking any future upgrade call.

5. Message library structure

On EVM, ULN-302's send logic and receive logic are deployed as separate contracts at different addresses, allowing independent upgrades of each direction.

On Stellar, the entire ULN-302 lives in a single Soroban contract, with send and receive logic split across modules (`send_uln.rs`, `receive_uln.rs`) coordinated by `uln302.rs`. The library's type is advertised via a `MessageLibType` enum (`Send`, `Receive`, `SendAndReceive`). Both directions share the same deployment, so upgrading one direction necessarily upgrades both.

6. Fee model

On EVM, fees follow a **pull model** — `SendLib` accumulates fees in a `fees[worker]` mapping and workers withdraw separately, and the Treasury takes a basis-point cut at send time.

On Stellar, fees follow a **push model**. The library's `send()` call returns a `Vec<FeeRecipient>`, and the endpoint immediately transfers tokens to each recipient within the same transaction. There is no deferred accumulation and no separate withdrawal step.

The push model has a direct consequence. If any recipient's token trustline is missing or frozen, the entire `send()` call reverts. On EVM, a fee-accumulation failure would not block the send. It would simply fail the later withdrawal.

The implementation mitigates this risk for the treasury ZRO fee by fixing the recipient to the treasury contract address itself, rather than allowing a configurable account. A contract address is always a valid token recipient in Soroban, whereas a configurable account address could lack the necessary trustline and cause every send to revert.

7. Worker (Executor/DVN) design

This section discusses the worker (Executor/DVN) design.

7.1 Contract structure

On EVM, the executor is a single upgradable contract (`Executor.sol`) that handles both fee quoting (called by the ULN during the send flow) and message execution (called by the off-chain executor

node on the destination chain). Both responsibilities live in the same deployed address.

On Stellar, these responsibilities are split across two separate contracts.

1. **LzExecutor**, which handles job assignment and fee calculation. The ULN calls `assign_job()` on this contract during the send flow, and it returns a `FeeRecipient` struct pointing to the deposit address.
2. **ExecutorHelper**, which handles actual message delivery. The off-chain executor node calls `ExecutorHelper.execute()` to deliver a message to an OApp.

7.2 Execution call direction

On EVM, message delivery routes through the endpoint. The executor calls `endpoint.lzReceive()`, which in turn clears the payload hash (advancing the inbound nonce) and then calls the OApp's `lzReceive()` callback. The endpoint sits between the executor and the OApp.

```
Executor.execute302()  
- endpoint.lzReceive()  
  - _clearPayload() // External call to endpoint to update nonce state.  
                    // On stellar, this reentrancy is not allowed.  
  - OApp.lzReceive() // External call to OApp callback
```

On Stellar, delivery bypasses the endpoint and calls the OApp directly, and `ExecutorHelper.execute()` invokes `lz_receive()` on the OApp receiver contract. The OApp is then responsible for calling `endpoint.clear()` to clear its own payload hash.

```
ExecutorHelper.execute()  
- OApp.lz_receive()  
  - endpoint.clear() // External call to endpoint to update nonce  
state  
  - __lz_receive() // Internal call to OApp callback
```

This inversion of the call direction is a consequence of Soroban's prevention of reentrancy. On EVM, the endpoint calls the OApp callback, and the OApp can call back into the endpoint (e.g., to send a composed message) because EVM allows reentrant calls (unless explicitly guarded). On Soroban, a contract that is already on the call stack cannot be entered again in the same transaction. If the execution were routed through the endpoint as on EVM, the OApp's `lz_receive()` callback would need to call `endpoint.clear()` to finalize delivery, which would be a reentrant call into the endpoint and would be rejected. By having the `ExecutorHelper` call the OApp directly, the endpoint is not on the call stack when the OApp executes, so the OApp can safely call `endpoint.clear()` to clear its own payload hash.

7.3 DVN as abstract account

On EVM, multi-sig-authorized DVN actions are executed by verifying M-of-N signatures. Self-configuration follows a self-call pattern. The DVN verifies signatures, then calls itself to apply the configuration change.

On Stellar, a contract cannot call itself within the same transaction (reentrancy prohibition). The self-call pattern is therefore unavailable. The Stellar DVN solves this by implementing Soroban's `CustomAccountInterface`, making the contract an abstract account. Instead of verifying signatures and performing an action in sequence inside a function body, the DVN has its own authorization logic in `__check_auth`. Soroban's runtime invokes `__check_auth` automatically before executing any call that requires the DVN's authorization.

7.4 Executor as abstract account and ABA messaging

On EVM, the executor calls the endpoint using `{gas: gasLimit}` syntax to cap the OApp callback's gas consumption.

On Stellar, the executor also implements `CustomAccountInterface` for a different reason to support ABA messaging. ABA is a pattern where an OApp on the destination chain (B) sends a response message back to the source chain (A) within the same `lz_receive` transaction.

On EVM this works because the endpoint calling the OApp, and the OApp calling back into the endpoint, are both allowed (EVM permits reentrant calls unless explicitly guarded).

On Stellar, the EVM approach would fail. If the executor called the endpoint, and the endpoint called the OApp, and the OApp tried to call the endpoint again to send a response, the second entry into the endpoint would be rejected as a reentrant call.

By implementing the executor as an abstract account, message delivery does not route through the endpoint at all. Instead, the executor authorizes the OApp's `lz_receive` invocation directly. The executor is not placed onto the call stack as a caller. It is present only as an authorizer. When the OApp subsequently calls `endpoint.send()`, which in turn calls `executor.assign_job()` to compute fees, the executor contract is accessible because it is not currently executing.

The executor's `__check_auth` validates the authorization context to restrict what the executor can authorize. Only `lz_receive`, `lz_compose`, `lz_receive_alert`, and `lz_compose_alert` calls are permitted. Alert calls must target the endpoint directly. Non-alert calls must target an OApp. This prevents the executor's authorization from being used to call arbitrary contracts.

7.5 Gas limit

On EVM, the executor explicitly caps the gas available to the OApp callback using `{gas: _executionParams.gasLimit}`. This prevents a malicious or buggy OApp from consuming all transaction gas.

On Stellar, Soroban has no gas-limit mechanism at the subcall level. The entire transaction runs

within the compute unit budget set by the submitter. The `gas_limit` field in `ExecutionParams` is carried as a parameter but is never used during execution. It is only included in the `lz_receive_alert` payload.

8. Encoding and resolution of bytes32 address

LayerZero's cross-chain protocol encodes all addresses as 32 bytes. On EVM, this is natural because Ethereum addresses are 20 bytes and can be zero-padded. On Stellar, addresses are `StrKey`-encoded structures of varying internal layout. A G-address (EOA) embeds a 32-byte Ed25519 public key, a C-address (contract) embeds a 32-byte contract ID hash, and an M-address embeds a G-address and a 64-bit ID and therefore cannot fit in 32 bytes. M-addresses are not used by the protocol.

The key observation is that G-addresses and C-addresses both carry exactly 32 bytes of payload, and those payloads are guaranteed to be in disjoint spaces. The protocol encodes addresses by stripping the type prefix and writing only the 32-byte payload into the packet header.

On the send side, `write_address_payload()` in the packet encoder discards the address type and writes the raw 32-byte payload. The packet header therefore contains no information about whether the sender or receiver is an EOA or a contract — only the payload is transmitted. On the receive side, the ULN resolves the receiver's `BytesN<32>` by treating it unconditionally as a contract ID.

The implementation applies a contract-first strategy. It first attempts to interpret the 32-byte value as a C-address and checks whether a contract is deployed at that address. If a contract exists, the address is treated as a C-address. If not, the value is interpreted as a G-address. This resolution is deterministic and unambiguous because the two address spaces do not overlap.

6. Assessment Results

During our assessment on the scoped LayerZero Stellar files, we discovered three findings. No critical issues were found. One finding was of high impact, one was of medium impact, and the remaining finding was informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.